



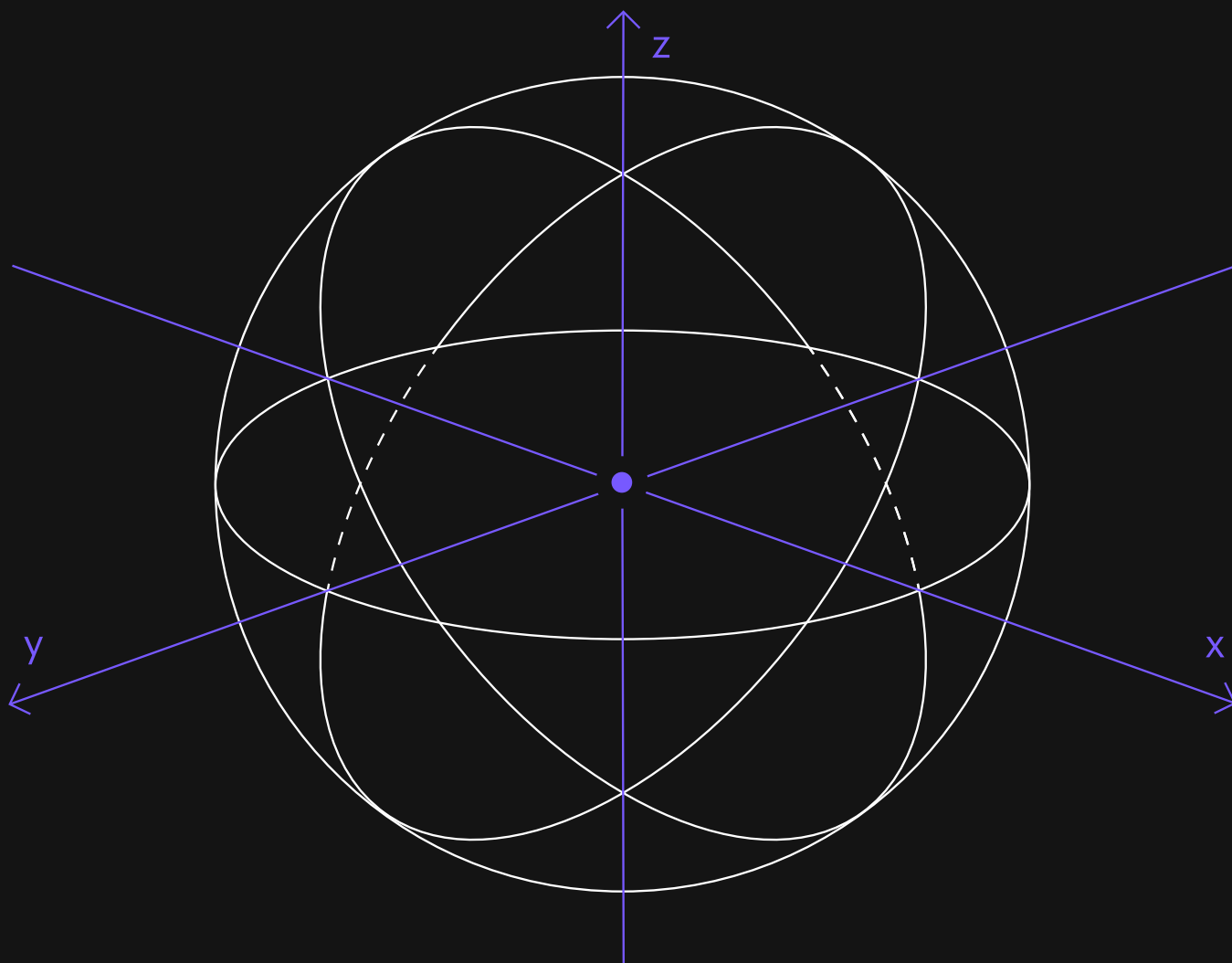
Машинное обучение

Лонгрид

Глубокое обучение

01

Введение в DL



Содержание

1. Зачем нужны нейронные сети	5
1.1. Классический подход к машинному обучению	5
1.2. Почему исходных признаков недостаточно	6
1.3. Представления, которые обучаются	7
1.4. Почему глубокое обучение стало практически успешным	9
2. От логистической регрессии к нейронной сети	9
2.1. Бинарная логистическая регрессия	9
2.2. Многоклассовый случай	11
2.3. Почему начинать стоит с простой модели	12
3. Многослойный персептрон	13
3.1. Композиция преобразований	13
3.1.1. Алгоритм прямого прохода	15
3.2. Иерархия признаков	16
3.3. Зачем нужна нелинейность	17
3.4. Число параметров	17
3.5. Переиспользование параметров	17
4. Универсальная аппроксимация	18
4.1. Ступенчатое приближение	19
4.2. Приближение индикатора полупрямой	19
4.3. Индикатор интервала	20
4.4. Строгая оценка ошибки для одного скрытого слоя	22
4.5. Чего теорема не гарантирует	23
5. Основные выводы	24

Лекция 1. Введение в глубокое обучение.

От логистической регрессии к многослойному персептрону

О чём эта лекция

Лекция связывает классические линейные модели с многослойными нейронными сетями. Мы последовательно разберём логистическую и мультиномиальную регрессию, устройство MLP, обучение представлений, гипотезу компактности, прямой проход, рост числа параметров, универсальную аппроксимацию, причины практического успеха глубокого обучения.

Организационные замечания

Курс по нейронным сетям является логическим продолжением курса машинного обучения. При этом нейронные сети будут рассматриваться не только как инструмент классификации или регрессии. По существу, это универсальный способ строить параметрические модели и обучать их по данным, поэтому область их применения значительно шире этих двух классических постановок.

В рамках курса будут разобраны базовые архитектуры нейронных сетей и их применение к разным типам данных: изображениям, текстам, звуку и, возможно, другим предметным областям.



Основные обозначения

- $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ — обучающая выборка из N объектов;
- $x_i \in \mathbb{R}^d$ — i -й объект, d — размерность входа;
- для фиксированного объекта x символ x_j обозначает его j -ю компоненту, $j = 1, \dots, d$;
- y_i — правильная метка i -го объекта: в бинарной задаче $y_i \in \{-1, +1\}$ (либо используется кодировка $\{0, 1\}$), а в многоклассовой — $y_i \in \{1, \dots, K\}$; для отдельного рассматриваемого объекта пишем y , K — число классов;
- $W^{(\ell)}, b^{(\ell)}$ — веса и смещения ℓ -го слоя;
- $z^{(\ell)}$ — значения до функции активации;
- $a^{(\ell)}$ — активации слоя, φ_ℓ — его функция активации;
- n_ℓ — ширина ℓ -го слоя, L — число обучаемых слоёв;
- π_i — вероятность положительного класса для i -го объекта в бинарной модели;
- $p = (p_1, \dots, p_K)$ — вероятности классов в многоклассовой модели, p_y — вероятность правильного класса;
- $\mathcal{L}$ — функция потерь на одном объекте;
- Q — эмпирический риск на обучающей выборке;
- $\tilde{f}$ — кусочно-постоянное приближение функции f ;

- $\hat{f}$ — произвольная нейросетевая аппроксимация в формулировке теоремы;
- $\hat{f}_k^{\text{int}}$ — конструкция через индикаторы интервалов, $\hat{f}_k^{\text{tel}}$ — телескопическая конструкция с одним скрытым слоем; k задаёт крутизну перехода.

1. Зачем нужны нейронные сети

1.1. КЛАССИЧЕСКИЙ ПОДХОД К МАШИННОМУ ОБУЧЕНИЮ

В классической постановке машинного обучения объект описывается вектором признаков x , после чего к нему применяется алгоритм классификации или регрессии. При этом неявно предполагается, что выбранное признаковое пространство уже достаточно хорошо отражает структуру задачи:

- похожие объекты находятся близко друг к другу;
- объекты одного класса образуют относительно компактные области;
- объекты разных классов можно разделить сравнительно простой границей.

Если это предположение выполняется, линейная модель, метод ближайших соседей, дерево решений или другой стандартный алгоритм могут успешно решить задачу.

Однако исходное представление сложных данных обычно такими свойствами не обладает.



Гипотеза компактности

Семантически похожим объектам должны соответствовать близкие векторные представления, а объекты разных классов должны образовывать области, которые можно сравнительно просто разделить. Нейронная сеть особенно полезна тогда, когда это свойство не выполняется для сырых данных и подходящее представление нужно выучить.

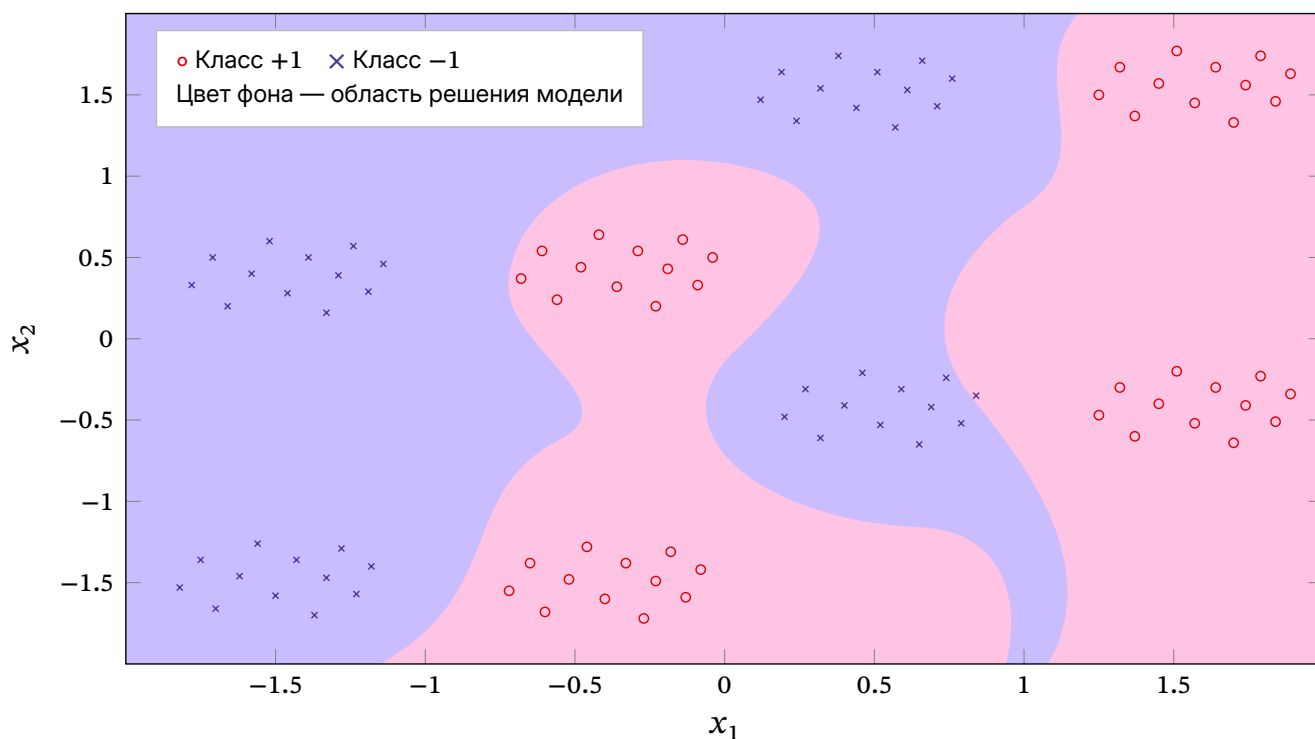


Рисунок 1. Пример признакового пространства, в котором локальная близость согласована с классами

1.2. ПОЧЕМУ ИСХОДНЫХ ПРИЗНАКОВ НЕДОСТАТОЧНО

Изображения. Изображение можно представить как матрицу яркостей пикселей, а затем вытянуть её в длинный вектор. Для монохромного изображения размера $H \times W$

$$x \in \mathbb{R}^{HW}.$$

Но небольшое смещение объекта сильно изменит этот вектор. Два представления одного и того же символа, расположенного в разных частях кадра, могут оказаться далеко друг от друга.

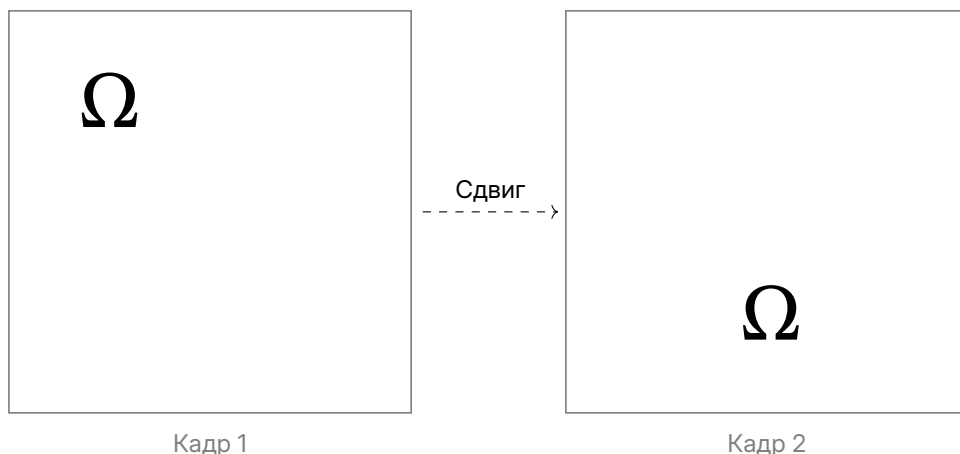


Рисунок 2. Один и тот же символ в двух разных положениях внутри кадра: содержание изображения не изменилось

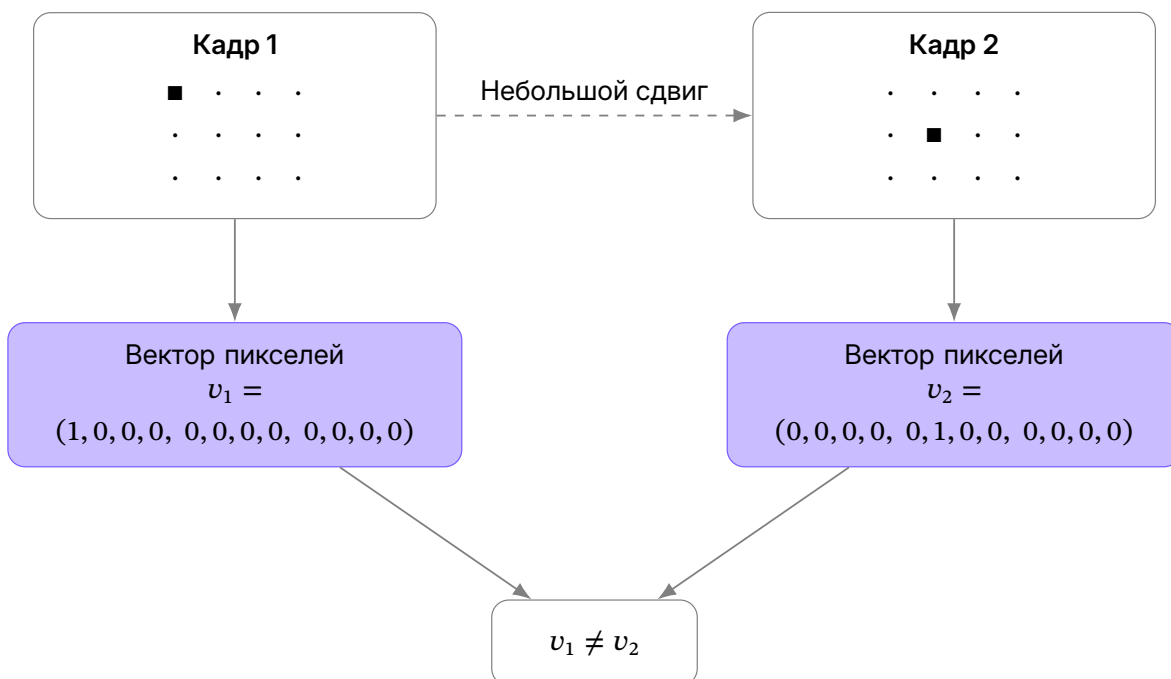


Рисунок 3. Смысл картинки сохраняется, но пиксельное представление изменяется: после сдвига один и тот же объект даёт разные векторы, $v_1 \neq v_2$

Следовательно, евклидова близость сырых пиксельных векторов не обязана совпадать с содержательной близостью изображений.

Текст. Текст можно представить набором символов, слов, n -грамм или частот. Но близость таких первичных признаков ещё не означает близость смысла. Два текста с похожими частотами слов могут относиться к разным темам, а два текста об одном и том же могут сильно различаться по лексике.

Например, предложения

«The cat is sleeping on the couch»,
«A feline rests upon the sofa»

имеют почти одинаковый смысл, но наборы слов для них практически не пересекаются. Их косинусное сходство в таком представлении может даже оказаться нулевым, несмотря на семантическую близость.

Звук. Звуковой сигнал в исходном виде представляет собой временной ряд значений амплитуды. Сдвиг сигнала по времени, изменение темпа или добавление шума могут значительно изменить числовое представление, хотя воспринимаемое нами содержание останется тем же.



Рисунок 4. Волновая форма аудиосигнала: её числовые значения чувствительны к голосу, темпу и громкости, хотя воспринимаемое содержание остаётся прежним

До распространения глубокого обучения такие задачи обычно решались в два этапа.

1. Специалист вручную конструировал информативные признаки.
2. На этих признаках обучался классификатор или регрессионная модель.

Например, в компьютерном зрении использовались гистограммы ориентированных градиентов (HOG), локальные бинарные шаблоны (LBP), SIFT-признаки и другие дескрипторы. Их цель состояла в том, чтобы превратить сырые пиксели в представление, более устойчивое к сдвигам, масштабу, освещению и другим несущественным изменениям.

1.3. ПРЕДСТАВЛЕНИЯ, КОТОРЫЕ ОБУЧАЮТСЯ

Ключевая идея глубокого обучения состоит в объединении двух этапов. Вместо того чтобы вручную задавать признаки, мы строим единую параметрическую модель, которая по обучающей выборке одновременно:

- формирует внутреннее представление данных;
- решает конечную задачу классификации, регрессии или предсказания.

Такое обучение называют **обучением представлений** (*representation learning*). Внутренние признаки не фиксируются заранее: они настраиваются вместе с остальными параметрами модели так, чтобы уменьшать выбранную функцию потерь.

Для изображений, текста и звука этот подход обычно даёт более высокое качество, поскольку модель может извлечь именно те признаки, которые нужны для конкретной задачи.

Это противопоставление можно изобразить двумя конвейерами.

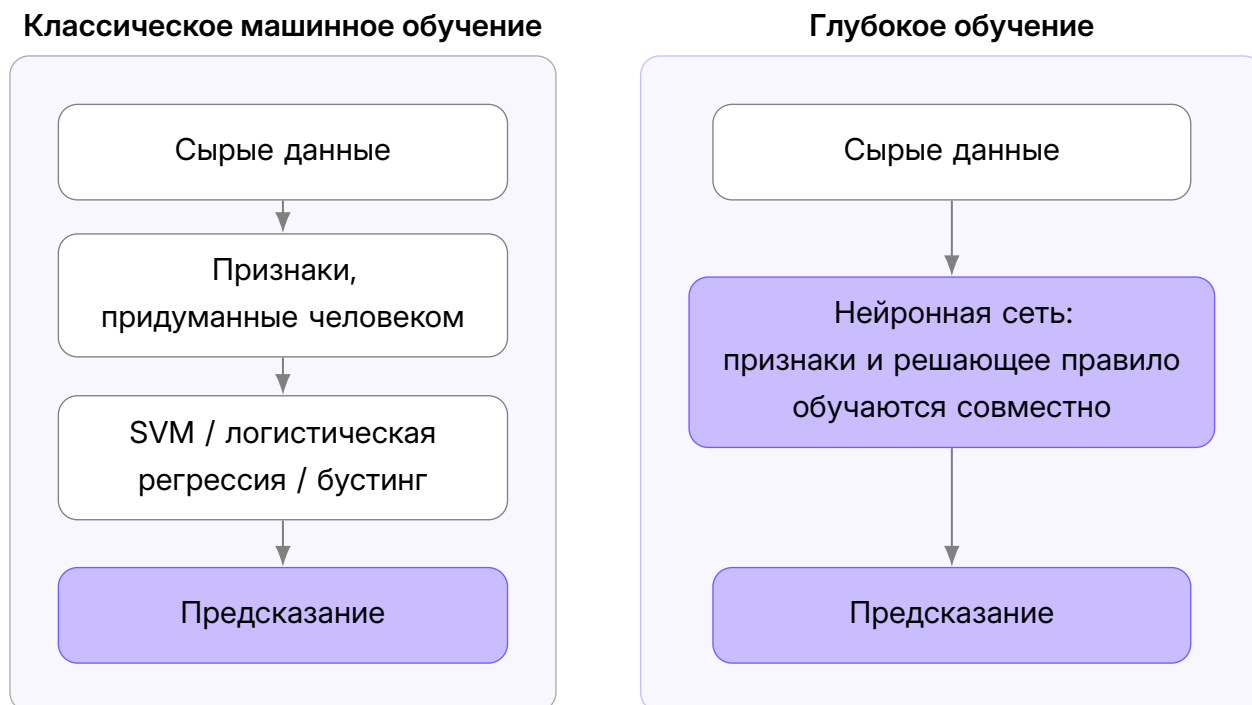


Рисунок 5. Два конвейера: в классическом машинном обучении признаки придумывает человек, в глубоком обучении они настраиваются вместе с решающим правилом

Эти же два подхода — на примере классификации изображения.

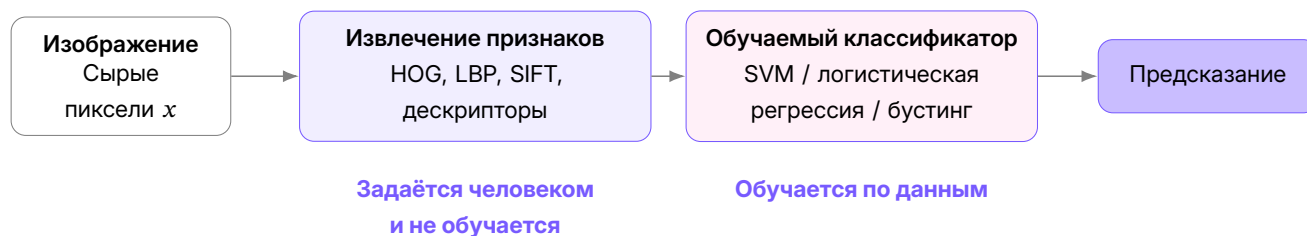


Рисунок 6. Классический подход: признаки проектирует человек, а по данным настраивается преимущественно финальный классификатор

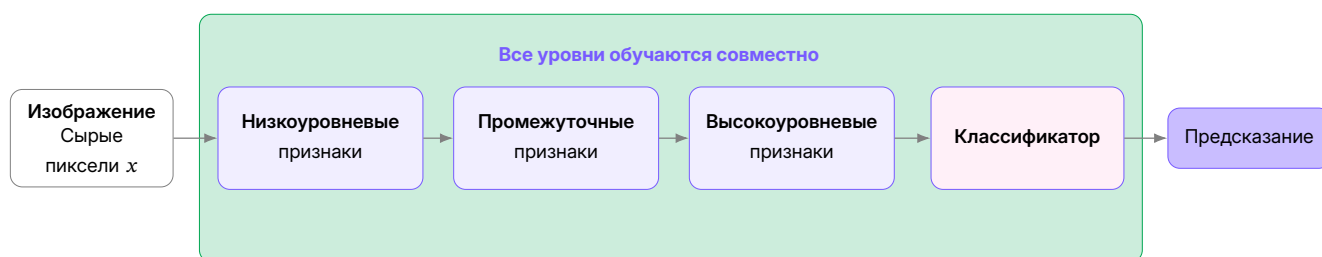


Рисунок 7. Глубокое обучение: низкоуровневые, промежуточные и высокоуровневые признаки настраиваются совместно с классификатором

1.4. ПОЧЕМУ ГЛУБОКОЕ ОБУЧЕНИЕ СТАЛО ПРАКТИЧЕСКИ УСПЕШНЫМ

Архитектурная идея обучения представлений соединилась с несколькими практическими факторами.

- **Матричные операции.** Основная работа слоёв сводится к умножению матриц, сложению векторов и покомпонентным операциям. Для таких вычислений существуют очень эффективные реализации.
- **GPU.** Графические процессоры выполняют большое число однотипных операций параллельно. Поэтому матричные вычисления нейронных сетей хорошо соответствуют современным ускорителям.
- **Большие объёмы данных.** Нейросети обладают высокой ёмкостью и способны описывать сложные зависимости, но для настройки множества параметров им обычно требуется много примеров. Рост доступных датасетов сделал такие модели практически полезными.

2. От логистической регрессии к нейронной сети

Пусть дана обучающая выборка:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad x_i \in \mathbb{R}^d.$$

В бинарной задаче $y_i \in \{-1, +1\}$, а в многоклассовой — $y_i \in \{1, \dots, K\}$.

2.1. БИНАРНАЯ ЛОГИСТИЧЕСКАЯ РЕГРЕССИЯ

Начнём с линейной модели. Для объекта $x \in \mathbb{R}^d$ вычисляется линейная оценка:

$$z = w^T x + b,$$

где $w \in \mathbb{R}^d$ — вектор весов, а b — свободный член.

Линейное решающее правило имеет следующий вид:

$$\hat{y}(x) = \text{sign}(w^T x + b).$$

В бинарной классификации эту оценку преобразуют в вероятность положительного класса с помощью сигмоиды:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad p(y = 1 \mid x, w, b) = \frac{1}{1 + \exp(-(w^T x + b))} = \sigma(w^T x + b).$$

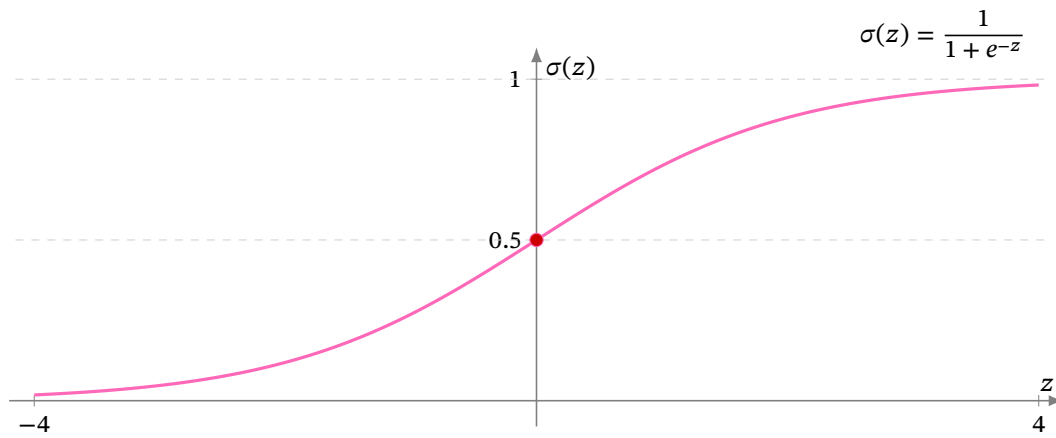


Рисунок 8. Сигмоида монотонно переводит линейную оценку $z \in \mathbb{R}$ в вероятность из интервала $(0, 1)$; точке $z = 0$ отвечает вероятность 0.5

Для меток $y \in \{-1, +1\}$ величина $\sigma(yz)$ — вероятность, которую модель приписывает правильной метке:

$$p(y = +1 | x, w, b) = \sigma(z), \quad p(y = -1 | x, w, b) = \sigma(-z).$$

Произведение yz называют отступом. Чем оно больше, тем увереннее правильное предсказание и тем меньше логистическая потеря. При уверенной ошибке $yz \ll 0$, поэтому значение потери быстро растёт.

Чем больше линейная оценка z , тем ближе вероятность класса 1 к единице. Для принятия решения вероятность сравнивают с порогом, часто равным 0.5.

Отдельный логистический нейрон можно представить так.

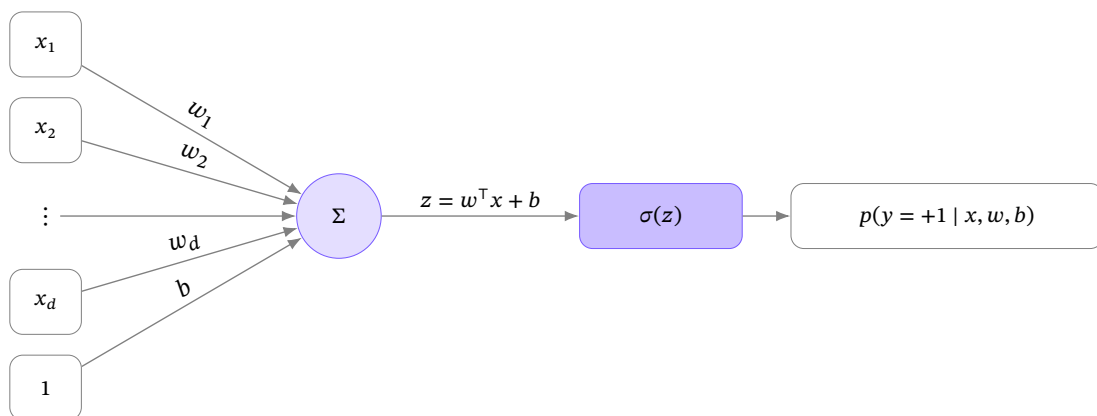


Рисунок 9. Логистический нейрон: взвешенная сумма входов со смещением b , а затем сигмоида, превращающая оценку в вероятность

Эту схему можно читать как **вычислительный граф**. Его вершины содержат входные, промежуточные и выходные значения, а стрелки показывают, какие операции необходимо выполнить. Для логистической регрессии прямое вычисление имеет последовательность

$$x \longrightarrow z = w^T x + b \longrightarrow \pi = \sigma(z) \longrightarrow \mathcal{L}.$$

Такое представление особенно удобно для нейросетей: большая модель строится как композиция тех же небольших функциональных блоков.

Если $y_i \in \{0, 1\}$, бинарная кросс-энтропия на одном объекте равна

$$\mathcal{L}_i(w, b) = -[y_i \log \pi_i + (1 - y_i) \log(1 - \pi_i)],$$

где $\pi_i = \sigma(w^\top x_i + b)$.

При метках $y_i \in \{-1, +1\}$ ту же потерю удобно записать как

$$\mathcal{L}_i(w, b) = \log(1 + \exp[-y_i(w^\top x_i + b)]).$$

Обозначая $z_i = w^\top x_i + b$, получаем эмпирический риск

$$Q(w, b) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i(w, b).$$

Иногда вместо среднего записывают сумму потерь. Эти варианты отличаются постоянным множителем и имеют одинаковые точки минимума.

2.2. МНОГОКЛАССОВЫЙ СЛУЧАЙ

Если имеется K классов, модель вычисляет отдельную линейную оценку для каждого класса:

$$z_k = w_k^\top x + b_k, \quad k = 1, \dots, K.$$

Если собрать веса и смещения в

$$W = \begin{bmatrix} w_1^\top \\ \vdots \\ w_K^\top \end{bmatrix} \in \mathbb{R}^{K \times d}, \quad b \in \mathbb{R}^K,$$

то все оценки классов вычисляются одновременно:

$$z = Wx + b.$$

Ненормированные оценки z_k принято называть **логитами** (*logits*).

Линейное решающее правило:

$$\hat{y}(x) = \arg \max_{k \in \{1, \dots, K\}} (w_k^\top x + b_k).$$

Все векторы w_k удобно объединить в матрицу W . Вероятности классов получают с помощью функции softmax:

$$p(y = k \mid x, W, b) = \frac{\exp(w_k^\top x + b_k)}{\sum_{j=1}^K \exp(w_j^\top x + b_j)}.$$

При этом

$$\sum_{k=1}^K p(y = k \mid x, W, b) = 1.$$

Это не K независимых бинарных моделей: вероятности всех классов связаны общим знаменателем. Объект относят к классу с максимальной вероятностью.

Обозначим матрицу объектов через $X = (x_1, \dots, x_N)$, а вектор правильных меток — через $\mathbf{y} = (y_1, \dots, y_N)$.

При условной независимости наблюдений правдоподобие всей выборки равно

$$p(\mathbf{y} | X, W, b) = \prod_{i=1}^N p(y_i | x_i, W, b).$$

Максимизация правдоподобия эквивалентна минимизации отрицательного логарифма:

$$\max_{W, b} p(\mathbf{y} | X, W, b) \iff \min_{W, b} [-\log p(\mathbf{y} | X, W, b)],$$

$$-\log p(\mathbf{y} | X, W, b) = -\sum_{i=1}^N \log p(y_i | x_i, W, b).$$

Это и есть многоклассовая кросс-энтропия.

Для одного объекта удобно использовать векторную запись:

$$z = Wx + b, \quad p = \text{softmax}(z),$$

$$\mathcal{L} = -\log p_y = -z_y + \log \sum_{k=1}^K e^{z_k}.$$

Вычислительный граф многоклассовой модели выглядит так.

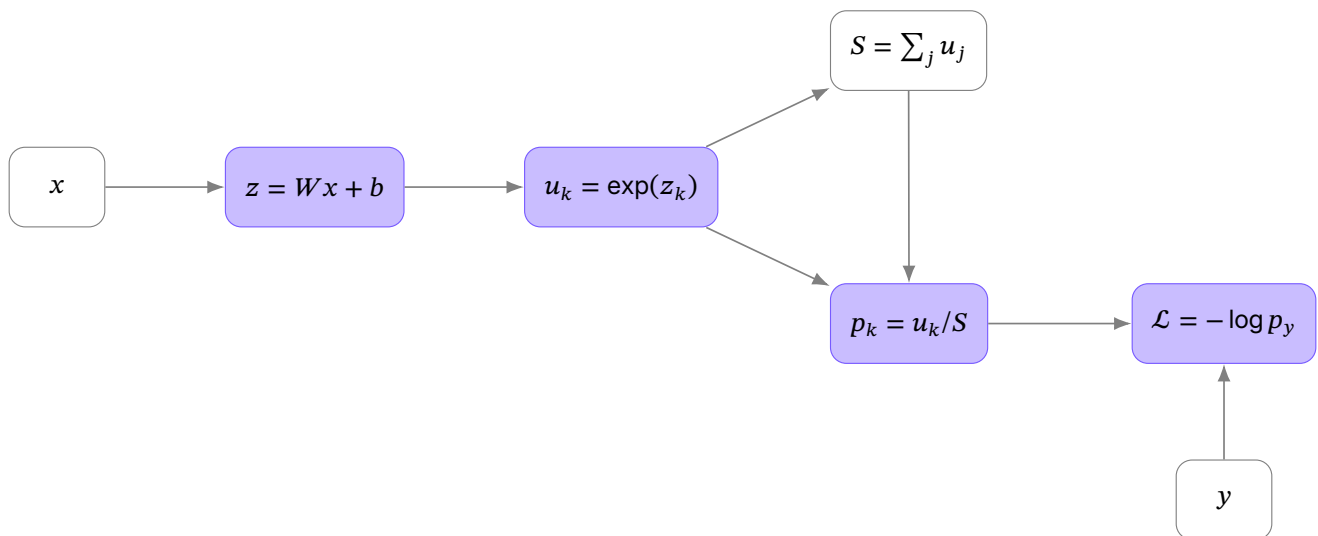


Рисунок 10. Вычислительный граф многоклассовой модели: правильная метка y подключается только на последнем шаге

Здесь метка y не участвует в вычислении логитов и вероятностей. Она нужна только в конце, чтобы выбрать p_y и оценить ошибку модели.

2.3. ПОЧЕМУ НАЧИНАТЬ СТОИТ С ПРОСТОЙ МОДЕЛИ

Линейные модели остаются полезными и популярными. Они быстро обучаются, требуют мало памяти, легко интерпретируются и нередко хорошо обобщают. Поэтому практическое решение задачи обычно разумно начинать с линейного базового варианта. Усложнение модели оправдано только тогда, когда простая модель не выражает важные нелинейные зависимости и более сложный подход действительно улучшает качество на отложенных данных.

3. Многослойный персептрон

3.1. КОМПОЗИЦИЯ ПРЕОБРАЗОВАНИЙ

Логистическая регрессия строит единственную линейную границу. Чтобы моделировать нелинейные зависимости, объединим несколько преобразований в слои.

Положим $a^{(0)} := x$. Каждый скрытый слой $\ell = 1, \dots, L - 1$ вычисляет

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)},$$

$$a^{(\ell)} = \varphi_{\ell}(z^{(\ell)}),$$

- где $W^{(\ell)}$ — матрица весов слоя;
- $b^{(\ell)}$ — вектор смещений;
- φ_{ℓ} — нелинейная функция активации, применяемая покомпонентно;
- $z^{(\ell)}$ — вектор до нелинейности;
- $a^{(\ell)}$ — вектор активаций слоя.

Последний слой преобразует полученное представление в требуемый выход: число, вектор оценок классов, распределение вероятностей и т. п.

Процесс последовательного вычисления слоёв от входа к выходу называется **прямым проходом** (*forward pass*).

В многоклассовой задаче выходной слой отдельно вычисляет логиты и вероятности:

$$z^{(L)} = W^{(L)} a^{(L-1)} + b^{(L)}, \quad p = \text{softmax}(z^{(L)}), \quad \mathcal{L} = -\log p_y.$$

Схема полносвязного MLP выглядит так.

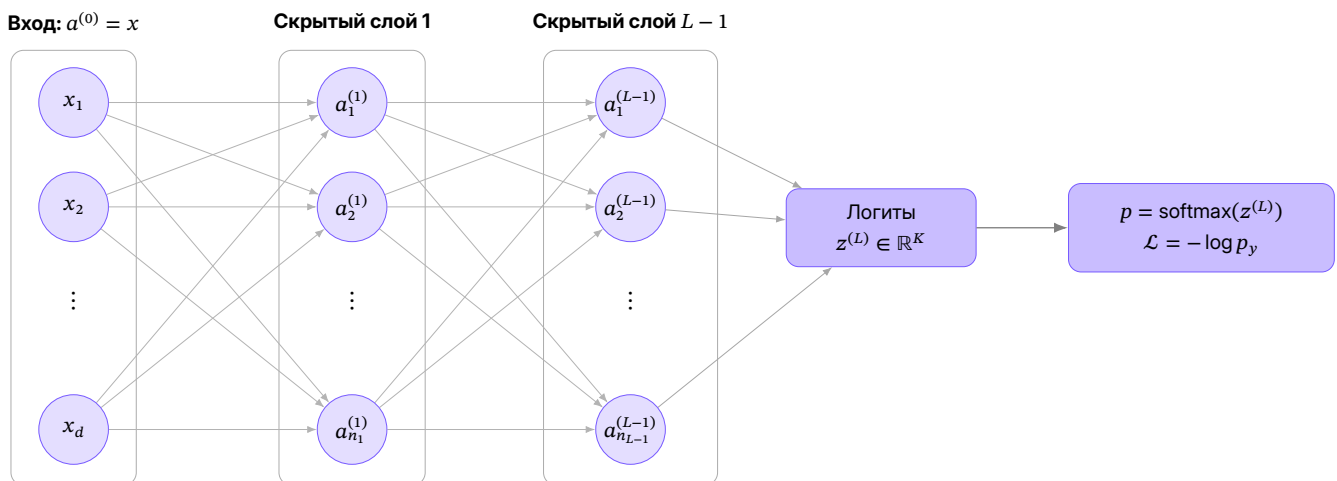
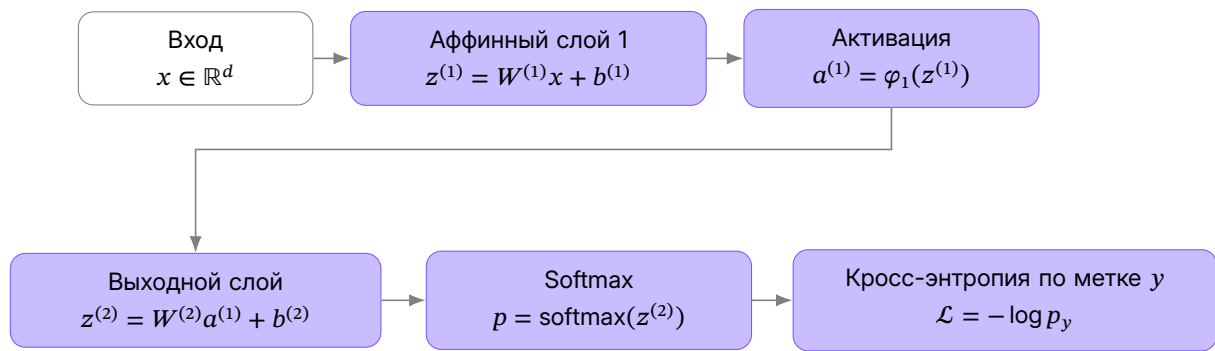


Рисунок 11. Схематическое изображение полносвязного MLP: показаны первый и последний скрытые слои, промежуточные слои по глубине не изображены.

Ту же идею для конкретной двухслойной сети удобно представить иначе — не как набор отдельных нейронов, а как цепочку матричных операций.

Рисунок 12. Двухслойная сеть как последовательность матричных операций: от входа x до значения функции потерь

Для сети с одним скрытым слоем, размером входа d , шириной скрытого слоя n и K классами размерности таковы.

Объект	Размерность	Смысл
$W^{(1)}$	$n \times d$	Веса первого аффинного преобразования
$b^{(1)}$	n	Смещения первого слоя
$z^{(1)}$	n	Преактивации скрытого слоя
$a^{(1)}$	n	Активации скрытого слоя
$W^{(2)}$	$K \times n$	Веса выходного преобразования
$b^{(2)}$	K	Смещения выходного слоя
$z^{(2)}$	K	Логиты классов
p	K	Вероятности классов

Таблица 1. Размерности величин в сети с одним скрытым слоем

ОПРЕДЕЛЕНИЕ

«Линейным слоем» в нейросетях обычно называют обучаемое аффинное преобразование $a \mapsto Wa + b$. «Слой активации» — покомпонентное применение нелинейной функции. Строго говоря, это операции, но такая терминология стандартна.

Скрытый нейрон может реагировать на некоторый шаблон в данных, однако он не является самостоятельным классификатором, а его выход не обязан иметь вероятностную интерпретацию.

Для одного слоя в пакетной записи, если столбцы матрицы $A^{(\ell-1)}$ соответствуют объектам:

$$Z^{(\ell)} = W^{(\ell)}A^{(\ell-1)} + b^{(\ell)}\mathbf{1}^T, \quad A^{(\ell)} = \varphi_\ell(Z^{(\ell)}).$$

В многоклассовой классификации последний слой обычно выдаёт логиты $z^{(L)}$, а затем применяется softmax:

$$p_k = \frac{\exp(z_k^{(L)})}{\sum_{j=1}^K \exp(z_j^{(L)})}.$$

Если e_y — one-hot-вектор правильного класса $y \in \{1, \dots, K\}$, то потеря для одного объекта равна

$$\mathcal{L} = - \sum_{k=1}^K (e_y)_k \log p_k = -\log p_y.$$

3.1.1. Алгоритм прямого прохода

Для классификационной MLP прямой проход можно записать единым алгоритмом. Пусть

$$n_0 = d, \quad n_1, \dots, n_{L-1}, \quad n_L = K,$$

$$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}, \quad b^{(\ell)} \in \mathbb{R}^{n_\ell}.$$

1. Тогда положить $a^{(0)} = x$.
2. Для скрытых слоёв $\ell = 1, \dots, L-1$ вычислить

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \varphi_\ell(z^{(\ell)}).$$

3. Для выходного слоя вычислить логиты

$$z^{(L)} = W^{(L)} a^{(L-1)} + b^{(L)}.$$

4. Получить вероятности и потерю

$$p = \text{softmax}(z^{(L)}), \quad \mathcal{L} = -\log p_y.$$



Предсказание и обучение

Для предсказания достаточно входа x : результатом будут логиты $z^{(L)}$, вероятности p или класс $\hat{y}$. Правильная метка y нужна при обучении, чтобы вычислить $\mathcal{L}$.

Для одного обучаемого слоя композиция имеет вид

$$\mathcal{L} = \text{CE}(Wx + b, y),$$

а для двух слоёв —

$$\mathcal{L} = \text{CE}(W^{(2)} \varphi_1(W^{(1)}x + b^{(1)}) + b^{(2)}, y).$$

Прямой проход удобно представить как последовательность функциональных элементов.

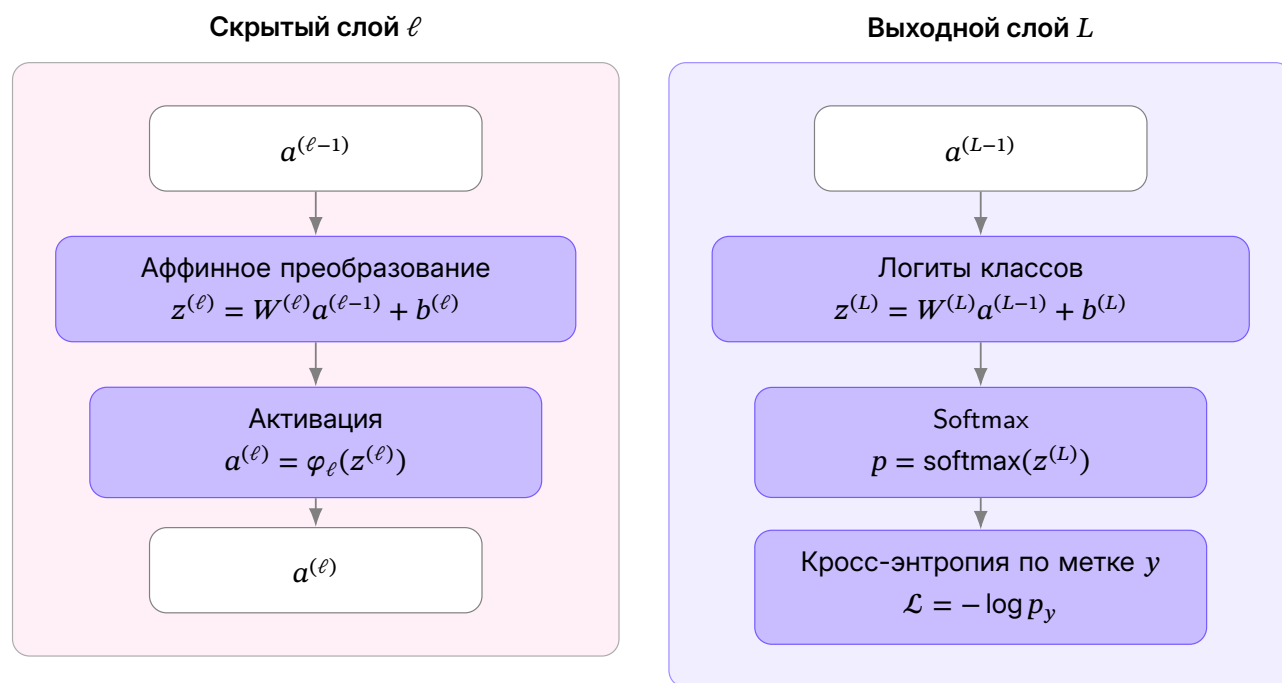


Рисунок 13. Устройство скрытого и выходного слоёв: аффинное преобразование, нелинейность и итоговая функция потерь

На схеме:

- блок аффинного преобразования вычисляет $z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}$;
- φ_{ℓ} — функция активации скрытого слоя;
- СЕ — выходная кросс-энтропия, включающая softmax и отрицательный логарифм вероятности правильного класса.

3.2. ИЕРАРХИЯ ПРИЗНАКОВ

Каждый слой можно интерпретировать как построение нового признакового пространства. В задаче распознавания изображений

$$x = a^{(0)} \longrightarrow a^{(1)} \longrightarrow a^{(2)} \longrightarrow \dots \longrightarrow a^{(L-1)} \longrightarrow z^{(L)}.$$

- Нейроны первых слоёв могут реагировать на простые локальные элементы — линии, края и углы;
- следующие слои объединяют их в более сложные формы и фрагменты объектов;
- глубокие слои описывают части объектов и целые объекты.

Важно, что эти признаки не проектируются вручную. Они возникают в процессе обучения, так как помогают уменьшать конечную функцию потерь.

Эту иерархию не следует понимать буквально: отдельному нейрону заранее не назначают смысл «линия» или «часть объекта». Интерпретируемая реакция может возникнуть как результат совместной оптимизации всей модели. Слово **deep** относится именно к глубине этой композиции: модель последовательно строит несколько уровней представления, а не один фиксированный набор признаков.

3.3. ЗАЧЕМ НУЖНА НЕЛИНЕЙНОСТЬ

Если между аффинными слоями не применять функцию активации, вся сеть останется аффинной. Действительно,

$$W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)} = \underbrace{W^{(2)}W^{(1)}}_{\tilde{W}}x + \underbrace{W^{(2)}b^{(1)} + b^{(2)}}_{\tilde{b}}$$

можно свернуть в одно аффинное преобразование — $\tilde{W}x + \tilde{b}$. Увеличение числа таких слоёв не повысит выразительность модели. Именно нелинейные функции φ_ℓ делают многослойную сеть способной задавать сложные границы и преобразования.

3.4. ЧИСЛО ПАРАМЕТРОВ

Полносвязный слой соединяет каждый вход с каждым выходом. Если у слоя d_{in} входов и d_{out} выходов, то число весов равно

$$d_{\text{in}}d_{\text{out}},$$

не считая d_{out} смещений.

Для сети с ширинами $n_0, n_1, \dots, n_L$ полное число обучаемых параметров равно

$$P = \sum_{\ell=1}^L (n_\ell n_{\ell-1} + n_\ell).$$

Например, для архитектуры

$$10\,000 \longrightarrow 300 \longrightarrow 300 \longrightarrow K$$

получаем

$$\begin{aligned} P &= (10\,000 \times 300 + 300) + (300 \times 300 + 300) + (300K + K) \\ &= 3\,090\,600 + 301K. \end{aligned}$$

При фиксированной ширине добавление слоёв увеличивает число параметров примерно линейно по глубине, но увеличение самой ширины скрытых слоёв даёт квадратичный рост внутренних матриц весов.

Поэтому число параметров растёт очень быстро. Например, цветное изображение размером 300×300 содержит 270 000 чисел. Даже слой из нескольких тысяч нейронов потребовал бы сотни миллионов или миллиарды весов. Для хранения и обучения такой модели нужны значительные вычислительные ресурсы и большой объём данных.

3.5. ПЕРЕИСПОЛЬЗОВАНИЕ ПАРАМЕТРОВ

Полносвязный слой не учитывает внутреннюю структуру входа: он назначает отдельный вес каждой паре «входной признак — выходной нейрон». Для изображений и последовательностей это часто избыточно. Один и тот же локальный элемент может встретиться в разных областях изображения или разных позициях последовательности.

Вместо отдельного детектора для каждой позиции можно применять один и тот же набор параметров W к нескольким частям входа.

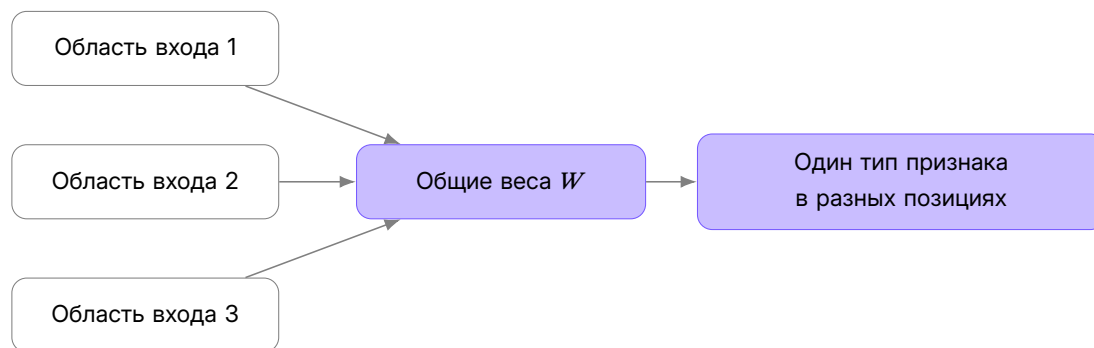


Рисунок 14. Разделение параметров: один и тот же набор весов W применяется к разным областям входа

Такое **разделение параметров** (*parameter sharing*) одновременно уменьшает число обучаемых весов и кодирует полезное предположение о структуре данных. Эта идея мотивирует специализированные архитектуры: свёрточные сети для пространственно локальных признаков, рекуррентные сети для последовательностей и трансформеры для взаимодействия элементов последовательности. Здесь это только мотивация; сами архитектуры рассматриваются далее в курсе.

4. Универсальная аппроксимация

Многослойный персептрон является универсальным аппроксиматором: при подходящем выборе нелинейности и достаточном количестве нейронов он может приблизить на компактном множестве любую непрерывную функцию с любой заранее заданной точностью.

ТЕОРЕМА

Одномерный вариант теоремы

Пусть f непрерывна на отрезке $[a, b]$, а непрерывная функция активации g удовлетворяет следующим условиям:

$$\lim_{u \rightarrow -\infty} g(u) = 0, \quad \lim_{u \rightarrow +\infty} g(u) = 1, \quad 0 \leq g(u) \leq 1.$$

Тогда для любого $\varepsilon > 0$ существует сеть с одним скрытым слоем, для которой

$$\sup_{x \in [a, b]} |f(x) - \hat{f}(x)| < \varepsilon.$$

Это утверждение описывает **выразительность** сети: оно гарантирует существование подходящих параметров, но пока ничего не говорит о том, как найти их обучением.

ТЕОРЕМА

Более общая формулировка

В конструктивном рассуждении ниже используется ограниченная sigmoid-подобная активация g : с ней удобно строить гладкие приближения порогов. В более общей теореме при стандартных условиях регулярности достаточно неполиномиальной функции активации. Универсальная аппроксимация справедлива и для ReLU.

4.1. СТУПЕНЧАТОЕ ПРИБЛИЖЕНИЕ

Идею можно понять на одномерном примере. Разобьём отрезок $[a, b]$ точками

$$a = a_0 < a_1 < \dots < a_m = b$$

и приблизим функцию константой на каждом маленьком интервале:

$$f(x) \approx \tilde{f}(x) = \sum_{i=0}^{m-2} f(\xi_i) \mathbb{I}[a_i \leq x < a_{i+1}] + f(\xi_{m-1}) \mathbb{I}[a_{m-1} \leq x \leq a_m], \quad \xi_i \in [a_i, a_{i+1}].$$

На следующем графике

- **линия** — исходная функция $f(x)$;
- **столбцы** — её кусочно-постоянное приближение $\tilde{f}(x)$;
- высота i -го столбца равна $f(\xi_i)$ на интервале $[a_i, a_{i+1}]$, а последний интервал замкнут справа.

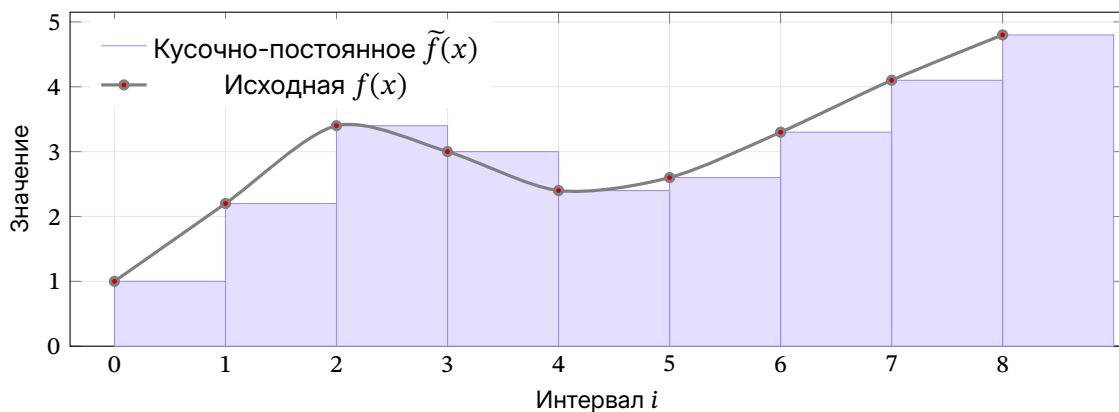


Рисунок 15. Функция $f(x)$ и её кусочно-постоянное приближение $\tilde{f}(x)$

Формула столбчатого приближения:

$$\tilde{f}(x) = \sum_{i=0}^{m-2} f(\xi_i) \mathbb{I}[a_i \leq x < a_{i+1}] + f(\xi_{m-1}) \mathbb{I}[a_{m-1} \leq x \leq a_m].$$

4.2. ПРИБЛИЖЕНИЕ ИНДИКАТОРА ПОЛУПРЯМОЙ

Для наглядной конструкции возьмём частный случай функции из теоремы: $g = \sigma$.

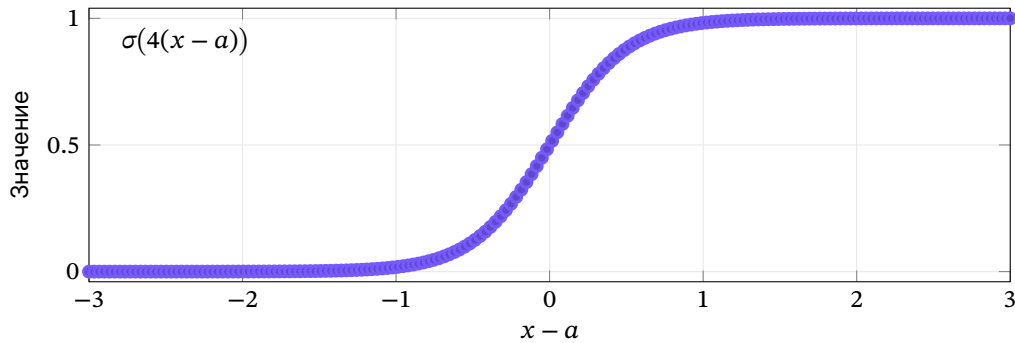
Индикатор условия $x \geq a$ можно приблизить сигмоидой с большим коэффициентом наклона:

$$\mathbb{I}[x \geq a] \approx \sigma(k(x - a)), \quad k \gg 1.$$

Чем больше k , тем резче переход около точки a :

$$\lim_{k \rightarrow \infty} \sigma(k(x - a)) = \begin{cases} 0, & x < a, \\ \frac{1}{2}, & x = a, \\ 1, & x > a. \end{cases}$$

То есть сходимость к индикатору имеет место поточечно для $x \neq a$, но не равномерно на множестве, содержащем точку a : в самой точке перехода сигмоида всегда равна $1/2$.

Рисунок 16. Сигмоида $\sigma(k(x - a))$ при $k = 4$ как гладкое приближение порога в точке a

На этом графике показана только одна кривая: $\sigma(4(x - a))$. Значения слева от a близки к 0, справа — к 1; при увеличении k переход становится ещё резче.

4.3. ИНДИКАТОР ИНТЕРВАЛА

Условие принадлежности интервалу является конъюнкцией двух условий:

$$\mathbb{I}[a \leq x \leq b] = \mathbb{I}[x \geq a] \wedge \mathbb{I}[x \leq b].$$

Если оба индикатора принимают значения 0 или 1, операцию «И» можно записать через сумму и порог:

$$\mathbb{I}[x \geq a] \wedge \mathbb{I}[x \leq b] = \mathbb{I}\left[\mathbb{I}[x \geq a] + \mathbb{I}[x \leq b] \geq \frac{3}{2}\right].$$

Заменяя каждый индикатор сигмоидой, получаем двухслойное приближение:

$$s_a(x) = \sigma(k(x - a)), \quad s_b(x) = \sigma(k(b - x)),$$

$$r_{a,b,k}(x) = \sigma\left(k\left[s_a(x) + s_b(x) - \frac{3}{2}\right]\right) \approx \mathbb{I}[a \leq x \leq b].$$

Более короткая конструкция использует разность двух сигмOID:

$$\mathbb{I}[a \leq x < b] \approx \sigma(k(x - a)) - \sigma(k(x - b)).$$

В концах интервала остаётся та же оговорка: непрерывная функция не может равномерно приблизить разрывный индикатор. Равномерное приближение исходной непрерывной f далее получается благодаря малости скачков ступенчатого приближения, а не из равномерной сходимости каждого индикатора.

Сетевая реализация конструкции с конъюнкцией

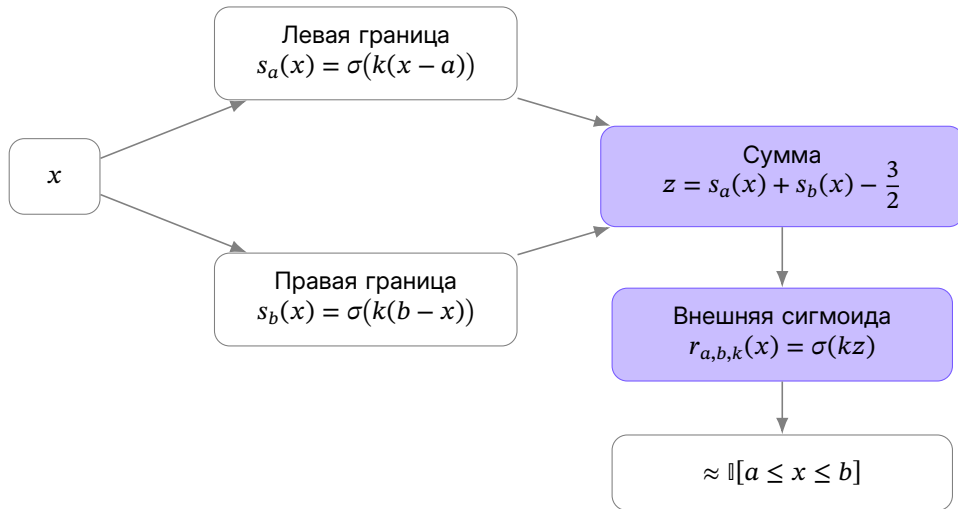


Рисунок 17. Индикатор интервала как двухслойная конструкция: две сигмоиды задают границы, а внешняя сигмоида играет роль логического «И»

Для каждого интервала положим

$$r_i(x) := r_{a_i, a_{i+1}, k}(x), \quad i = 0, \dots, m-1.$$

Последний линейный слой суммирует эти нейроны с коэффициентами $f(\xi_i)$:

$$\hat{f}_k^{\text{int}}(x) = \sum_{i=0}^{m-1} f(\xi_i) r_i(x) \approx \tilde{f}(x) \approx f(x).$$

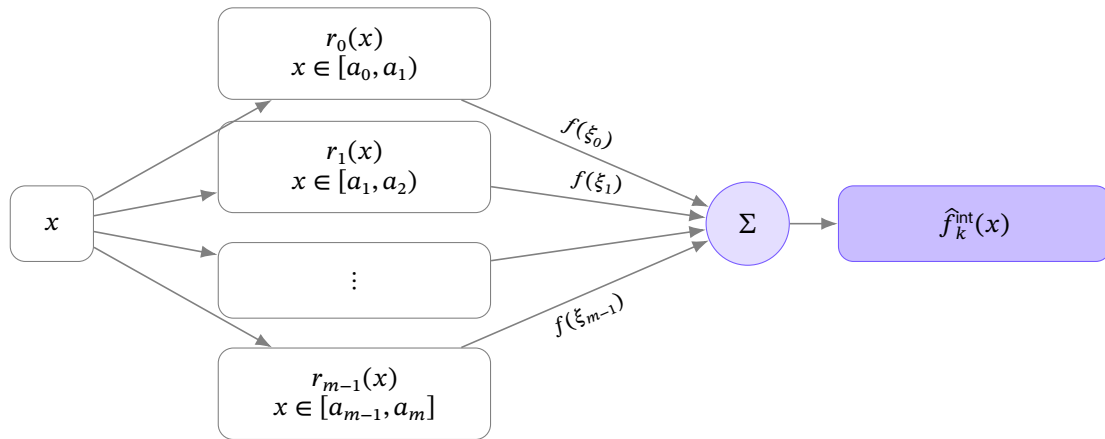


Рисунок 18. Сеть, приближающая f : каждый скрытый нейрон отвечает за свой интервал, а выходной слой складывает их с весами $f(\xi_i)$

Уменьшая ширину интервалов и увеличивая k , можно повышать точность приближения вдали от крайних точек; строгая равномерная оценка на всём отрезке будет получена ниже.

4.4. СТРОГАЯ ОЦЕНКА ОШИБКИ ДЛЯ ОДНОГО СКРЫТОГО СЛОЯ

Предыдущая конструкция через индикаторы интервалов наглядна. Теперь покажем телескопическую конструкцию, которая непосредственно даёт сеть с **одним** скрытым слоем.

Поскольку f непрерывна на компактном отрезке, она равномерно непрерывна. Вернёмся к разбиению из раздела 4.1., зафиксируем $\xi_i := a_i$ и для удобства переобозначим $t_i := a_i$:

$$a = t_0 < t_1 < \dots < t_m = b$$

Равномерная непрерывность позволяет выбрать это разбиение так, чтобы на каждом интервале колебание функции было меньше $\varepsilon/3$:

$$\sup_{u,v \in [t_{i-1}, t_i]} |f(u) - f(v)| < \frac{\varepsilon}{3}.$$

Обозначим

$$\Delta_i = f(t_i) - f(t_{i-1})$$

и введём функцию Хевисайда

$$H(u) = \begin{cases} 0, & u < 0, \\ 1, & u \geq 0. \end{cases}$$

Ступенчатое приближение

$$\tilde{f}(x) = f(t_0) + \sum_{i=1}^{m-1} \Delta_i H(x - t_i)$$

на интервале $[t_j, t_{j+1})$ равно $f(t_j)$. Поэтому

$$\sup_{x \in [a, b]} |f(x) - \tilde{f}(x)| < \frac{\varepsilon}{3}.$$

Заменяем каждый скачок сглаженной ступенькой:

$$g(k(x - t_i)), \quad k > 0,$$

$$\hat{f}_k^{\text{tel}}(x) = f(t_0) + \sum_{i=1}^{m-1} \Delta_i g(k(x - t_i)).$$

Из выбранного разбиения следует, что

$$|\Delta_i| < \frac{\varepsilon}{3}.$$

Положим

$$M = \sum_{i=1}^{m-1} |\Delta_i|, \quad \delta = \frac{\varepsilon}{6(1 + M)}.$$

Из предельных свойств g существует $\rho > 0$ такое, что

$$|g(u) - H(u)| < \delta \quad \text{при } |u| \geq \rho.$$

Выберем достаточно большое k :

$$k > \frac{2\rho}{\min_{0 \leq i < m} (t_{i+1} - t_i)}.$$

Тогда переходные интервалы $(t_i - \rho/k, t_i + \rho/k)$ не пересекаются. Для каждого фиксированного x не более одного нейрона находится в переходной области; его вклад в ошибку не превосходит $|\Delta_i| < \varepsilon/3$.

Для всех остальных нейронов ошибка хвоста в сумме меньше $\delta M < \varepsilon/6$. Следовательно,

$$\sup_{x \in [a,b]} |\tilde{f}(x) - \hat{f}_k^{\text{tel}}(x)| < \frac{\varepsilon}{2}.$$

По неравенству треугольника

$$\begin{aligned} \sup_{x \in [a,b]} |f(x) - \hat{f}_k^{\text{tel}}(x)| &\leq \sup_x |f(x) - \tilde{f}(x)| + \sup_x |\tilde{f}(x) - \hat{f}_k^{\text{tel}}(x)| \\ &< \frac{\varepsilon}{3} + \frac{\varepsilon}{2} = \frac{5\varepsilon}{6} < \varepsilon. \end{aligned}$$

Наконец,

$$g(k(x - t_i)) = g(w_i x + b_i), \quad w_i = k, \quad b_i = -kt_i.$$

Значит,

$$\hat{f}_k^{\text{tel}}(x) = f(t_0) + \sum_{i=1}^{m-1} \Delta_i g(w_i x + b_i)$$

— это сеть с одним скрытым слоем из $m - 1$ нейронов и линейным выходом. Выходные веса равны Δ_i , а выходное смещение — $f(t_0)$.

4.5. ЧЕГО ТЕОРЕМА НЕ ГАРАНТИРУЕТ

Однако из теоремы об универсальной аппроксимации не следует:

- что нужное приближение будет компактным;
- его параметры легко найти;
- для обучения потребуется разумное количество данных;
- модель будет хорошо работать на новых данных или за пределами рассматриваемого компакта.

В многомерном пространстве возникает проклятие размерности: прямое разбиение области на маленькие ячейки требует экспоненциально большого числа элементов. Поэтому формальная выразительность архитектуры ещё не гарантирует, что подходящая модель окажется компактной, легко обучаемой и способной хорошо обобщать.

5. Основные выводы

1. Бинарная логистическая регрессия объединяет аффинный логит, сигмоиду и логарифмическую функцию потерь.
2. Мультиномиальная регрессия вычисляет K логитов и совместно нормирует их функцией softmax.
3. Глубокое обучение объединяет построение признаков и конечное решающее правило в одной обучаемой модели. Цель внутренних преобразований можно связать с гипотезой компактности.
4. MLP чередует аффинные преобразования и покомпонентные нелинейности. Без нелинейностей вся композиция схлопывается в одно аффинное отображение.
5. Прямой проход последовательно вычисляет внутренние представления, логиты, вероятности классов и функцию потерь.
6. Теорема об универсальной аппроксимации гарантирует существование приближающей сети, но не лёгкость обучения, компактность модели или хорошее обобщение.
7. Число параметров полносвязной сети быстро растёт с шириной слоёв: это важная цена выразительности модели.
8. Переиспользование одних весов в разных позициях уменьшает число параметров и позволяет учитывать структуру изображений и последовательностей.
9. Практический успех DL связан с совместным обучением признаков, эффективными матричными вычислениями, GPU и большими объёмами данных.



Ключевые формулы

Бинарная логистическая регрессия

$$z = w^T x + b, \quad \mathcal{L} = -\log \sigma(yz).$$

Мультиномиальная регрессия

$$z = Wx + b, \quad p = \text{softmax}(z), \quad \mathcal{L} = -\log p_y.$$

Скрытый слой MLP

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \varphi_{\ell}(z^{(\ell)}).$$

Число параметров

$$P = \sum_{\ell=1}^L (n_{\ell} n_{\ell-1} + n_{\ell}).$$